

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
имени М.В. ЛОМОНОСОВА»**

ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

КУРСОВАЯ РАБОТА

**«Автоматический шаблонизатор
в брокере сообщений»**

Автор:

Нарембеков Темирлан,
магистрант 1 курса ВМК КФ МГУ

Научный руководитель:

Смирнов Илья Николаевич

Весна 2026

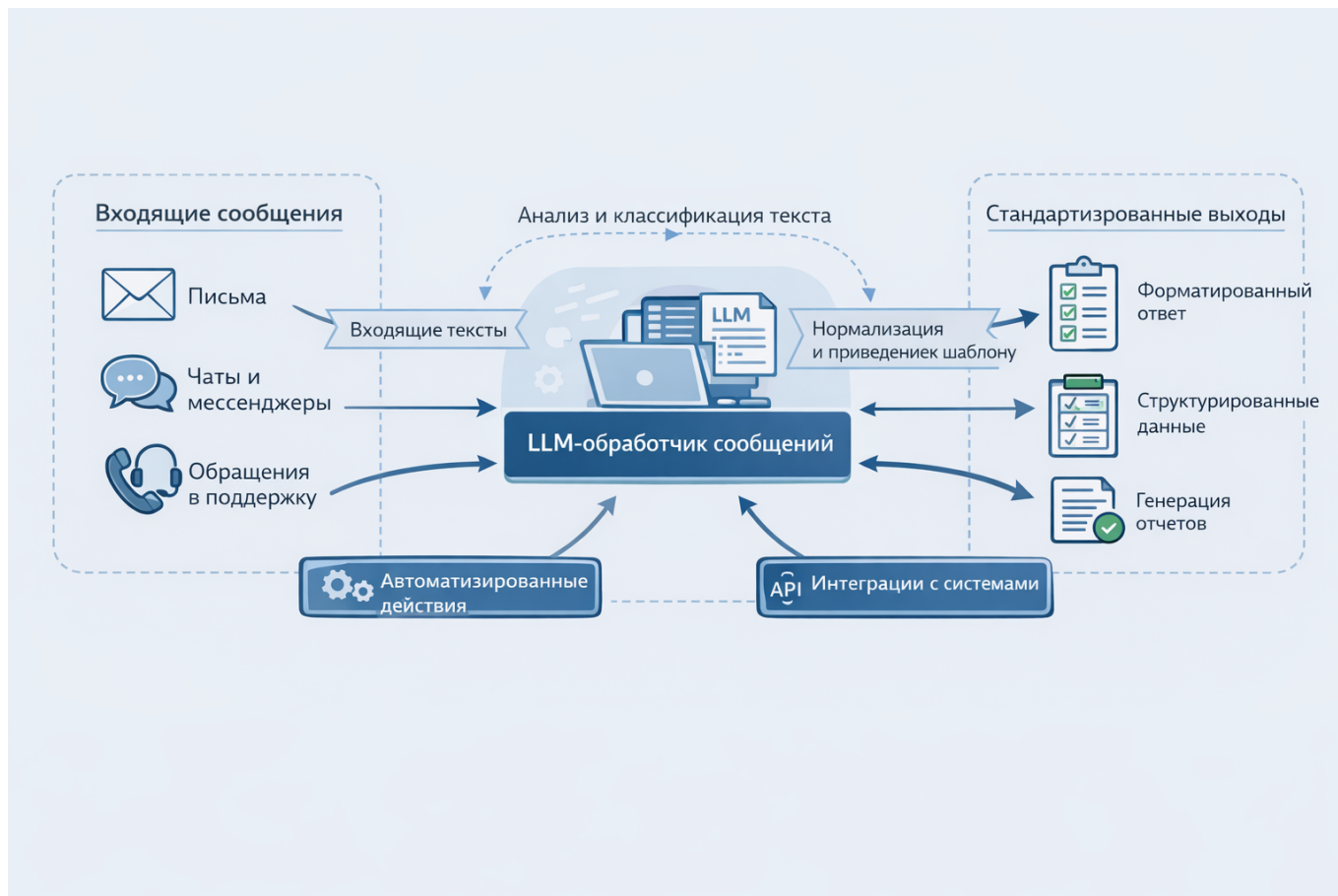
Содержание

1	Что хотим сделать	2
2	ОБЗОР СТАТЕЙ И ЛИТЕРАТУРЫ	2
2.1	"Брокеры сообщений в системах обработки данных" Селезнёв Александр Игоревич, ассистент; Селезнёв Игорь Львович, кандидат технических наук, доцент Белорусский государственный университет информатики и радиоэлектроники (г. Минск, Беларусь)	2
2.2	Обзор современных технологий извлечения знаний из текстовых сообщений А. А. Мусаева, Д. А. Григорьев	6
2.3	Drain: An Online Log Parsing Approach with Fixed Depth Tree	8
2.4	Сравнительный анализ 18 LLM моделей: конец монополии? ХАБР	12
3	АНАЛИЗ АНОМАЛИЙ	15
3.1	Данные	15
3.2	Разбиение данных по выборкам	16
3.3	Векторизация объектов	17
3.4	HDBSCAN	18
3.5	OPTUNA	18
3.6	Оптимизация по $F_{1/2}$	23
3.7	ОПТИМИЗАЦИЯ UMAP	23
4	Доверительное оценивание F_1	25

1 Что хотим сделать

Планируется сделать чатбот с интегрированным брокером сообщений со встроенной LLM моделью для обработки текста и его классификации с последующим приведением к шаблону(образцу).

Для этого проведем обзор статей с похожими задачами и статей для выбора LLM.



2 ОБЗОР СТАТЕЙ И ЛИТЕРАТУРЫ

2.1 "Брокеры сообщений в системах обработки данных"

Селезнёв Александр Игоревич, ассистент; Селезнёв Игорь Львович, кандидат технических наук, доцент Белорусский государственный университет информатики и радиоэлектроники (г. Минск, Беларусь)

Брокер сообщений — это программный компонент, служащий посредником в коммуникации между различными частями распределенной системы обработки информации. В этом случае брокер сообщений

реализуется как часть общей архитектуры системы, либо как отдельный сервис

Существуют два основных типа брокеров сообщений — с организацией взаимодействия производитель-потребитель **producer-consumer** и с организацией взаимодействия издатель-подписчик **publishersubscriber**

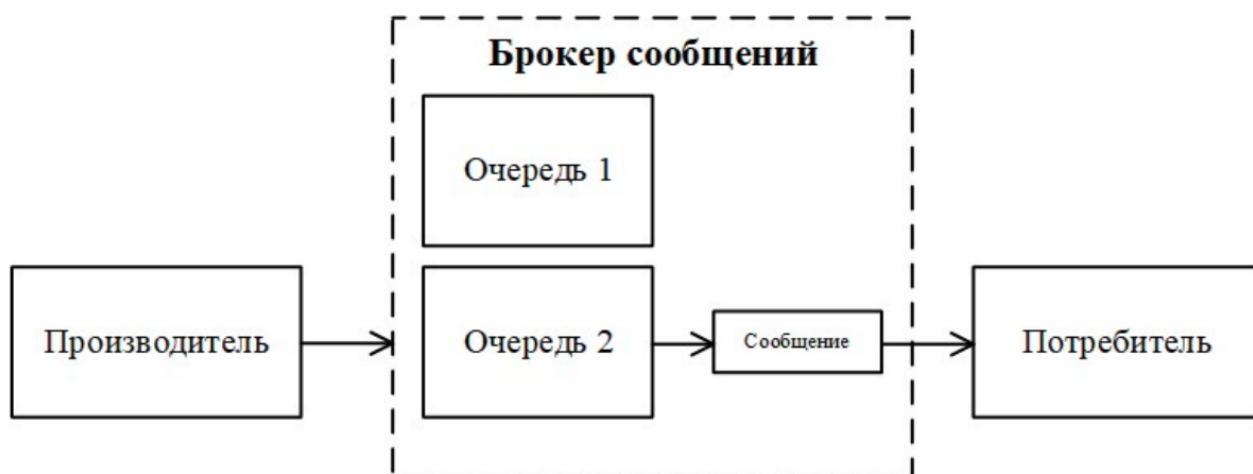


Рис. 1. Структура брокера сообщений с организацией взаимодействия производитель-потребитель

Из рисунка 1 видно, что «Производитель» записывает информацию в две очереди сообщений, затем выбирает «Сообщение» из второй очереди и посылает его получателю «Потребитель». Такая структура брокера сообщений характерна для брокеров, построенных на использовании очередей (queuebased).

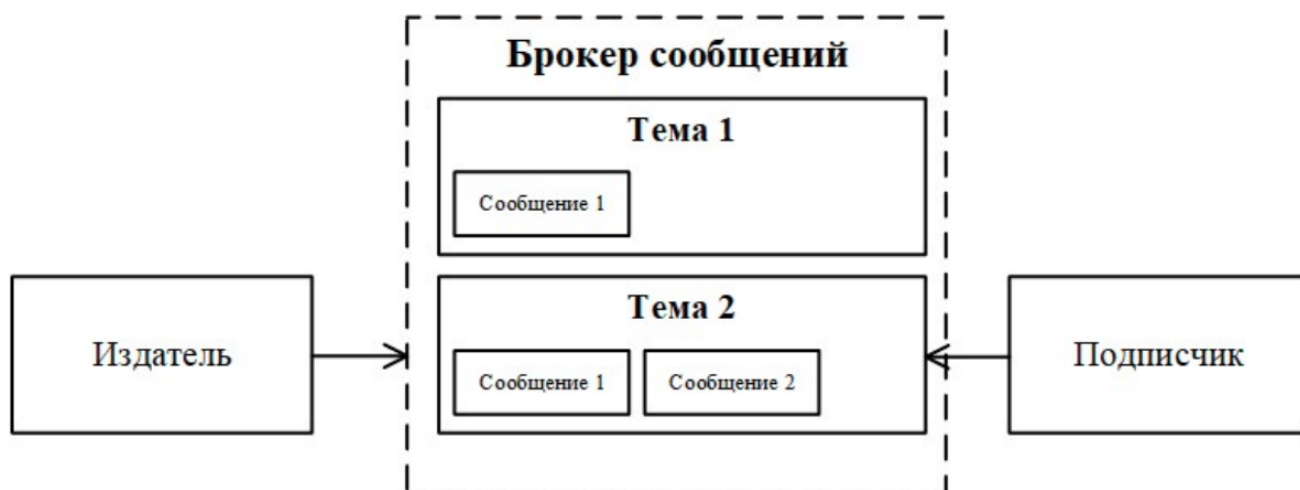


Рис. 2. Структура брокера сообщений с организацией взаимодействия издатель-подписчик

Из рисунка 2 видно, что «Издатель» производит публикацию сообщений с помощью двух тем, при этом в системе имеется потребитель «Подписчик», который «читает» вторую тему, получая возможность

просматривать «Сообщение 1» и «Сообщение 2» с помощью настроенных прав доступа для него. В этом случае у потребителя «Подписчик» нет доступа к первой теме. Такая структура брокера сообщений характерна для потоковых (streaming) брокеров сообщений.

Преимущества брокеров сообщений

1. **Отказоустойчивость и масштабируемость** - за счет возможности реализации кластеров брокеров
2. **Разделение слоев исполнения и интерфейсов взаимодействия**
3. **Гарантии доставки сообщений** - Информация сохраняется в очереди с возможностью отложенной обработки
4. **Повышение производительности** - за счет асинхронной обработки
5. **Безопасность обмена данными** - механизмы шифрования и аутентификации

Недостатки брокеров сообщений

1. **Усложнение системы** - за счет добавления нового элемента реализации кластеров брокеров
2. **Дополнительная нагрузка на сеть** - за счет увеличения объема передаваемых данных
3. **Возможность конфликтов версий:**

Наиболее популярные брокеры сообщений **RabbitMQ, Kafka, AWS SNS/ SQS**

Брокер сообщений RabbitMQ

Использует протокол очередей сообщений **AMQP** и имеет организацию взаимодействия **производитель-потребитель** в которую дополнительно включены блоки **обменника (exchange)** и **маршрутизации и хранения(routing-binding)** правил доступа для **exchange**

- I. производитель отправляет сообщения в брокер
- II. сообщения поступают в **обменник**
- III. после они попадают в **routing-binding** и по установленным правилам доступа передаются в нужные очереди сообщений
- IV. затем они уже направляются к **потребителю**

Конфигурации **exchange**:

- 1) **Маршрутизация без маршрутизации Fanout exchange** - отправка сообщений во все очереди
- 2) **Маршрутизация по ключу (Direct exchange)** - распределение сообщений по очередям по ключу маршрутизации
- 3) **Маршрутизация по шаблону (Topic exchange)** - на основе шаблонов заданных в ключе маршрутизации

Достоинства **RabbitMQ**:

- 1) Гибкая маршрутизация
- 2) простота развертывания на веб-серверах

Недостатки

- 1) Невозможность просмотра очередей
- 2) Замедление работы при высокой нагрузке - из-за кластера

Брокер сообщений Kafka

Функционал: Запись хранение и выдача по запросу с взаимодействием **издатель-подписчик**

- Новые сообщения добавляются в журнал фиксаций(логов) брокера
- Использует **Pull-механизм** в отличие от **RabbitMQ**
- Для чтения сообщения с произвольного места подписчик сообщает **номер сообщения и номер части** диска в котором он хранится

Достоинства **RabbitMQ**:

- 1) простота развертывания на веб-серверах
- 2) чтение в произвольном порядке и одновременно Недостатки

- 1) сложность с обработкой "испорченных" сообщений
- 2) Необходимость учета последнего сообщ прочитанного пользователем

Брокер сообщений Amazon Simple Queue Service

Функционал: два вида очередей **FIFO** и Стандартные очереди с нарушениями с взаимодействием **издатель-подписчик**

Достоинства **Amazon SQS**:

- 1) Популярность
- 2) Шифрование сообщений
- 2pt| 1) Резервное копирование
- 2) Интеграция с сервисами Amazon

Недостатки

- 1) Полная зависимость от AWS

Выбор брокера Выбор зависит от проектируемой системы обработки данных и выбранных критериев разработки

Для небольших СОД подойдет **RabbitMQ**, при больших **Kafka**

2.2 Обзор современных технологий извлечения знаний из текстовых сообщений

А. А. Мусаева, Д. А. Григорьев

Схема представляет из себя формальное описание данных в сообщении; она определяет поля и типы данных

Предобработка

В статье приводятся

Токенизация - представление текста в виде последовательности мелких частей (токены)

Стемминг - выделение и сохранение наиболее значимых частей слов

0. Промежуточный этап - очистка текста преобразующая неструктурированный текст в промежуточную форму (**IF**)

1. **Отбор признаков** - устранение ненужной информации из текста

РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ(Regular Expressions) - поиск точных вхождений подстроки в тексте по заданному шаблону. Применяется при очистке данных в токенизации, удалении стоп-слов

фреймворк *TokensRegex* из пакета Stanford CoreNLP - позволяет делать поиск шаблона не только по тексту но и по токенизации

Морфологический разбор - позволяет перейти от слов к их корням

ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ

Извлечение признаков состоит в преобразовании предобработанного текста в векторное цифровое представление пригодное для инференса машинного обучения

1) **Bag of Words** - Представление текста в виде вектора где каждая компонента соответствует одному из уникальных слов, а число в ней это кол-во раз когда это уникальное слово встретилось в тексте.

2) **Word2Vec** - представление слова в виде вектора.

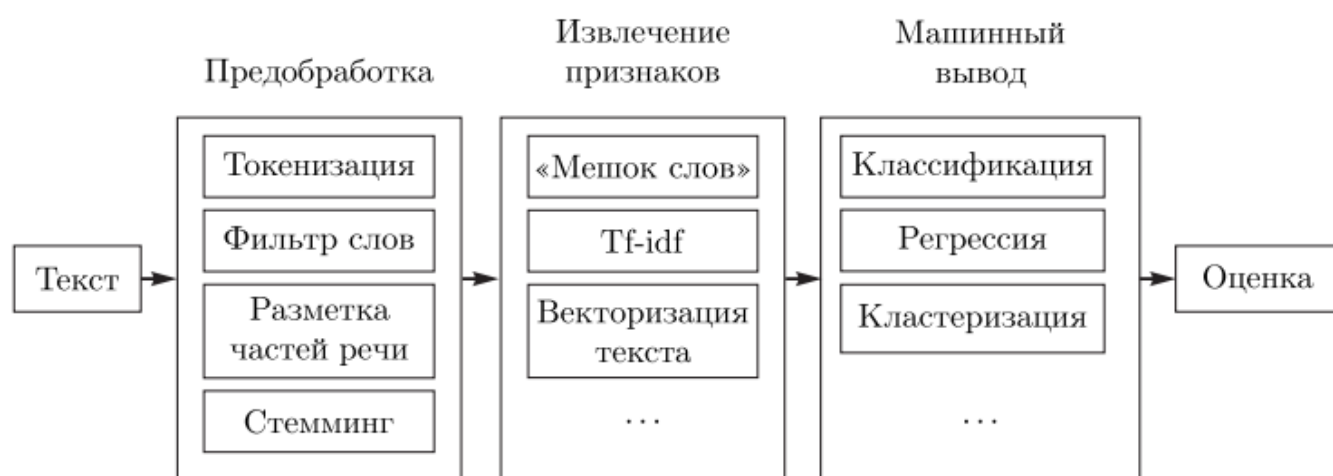


Рис. 2

Ppt

ПРОЧЕЕ

NLTK, GENSIM

Google Cloud Natural Language, IBM Watson NLP/NLU, Microsoft Text Analytics - инструменты для анализа документов на основе предобученных моделей с возможностью добавления данных для дообучения

2.3 Drain: An Online Log Parsing Approach with Fixed Depth Tree

В разработке и поддержке различных сервисов необходим анализ логирования(отчеты о состоянии ПО во время его выполнения). Для этого необходим парсинг логов.

Сырые данные в логировании программ как правило неструктурированы.

В этой статье предложен метод онлайн парсинга сырых логирований для структурирования логов и распределения их по своим группам. Для этого применяется дерево парсинга фиксированной глубины, в чьи листья закодированы правила парсинга

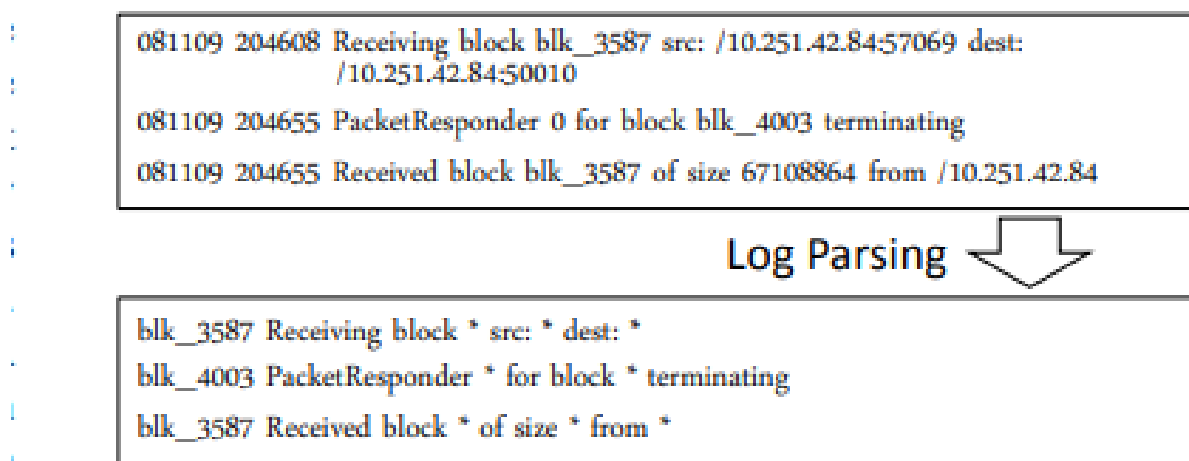


Fig. 1: Overview of Log Parsing

I. Предобработка входных сырых логов на основе специфичных regular expressions(например извлечение ID,IP). Эти выражения сверяют не сами сообщения логов а их токены.

II. Проход по дереву через посредством специальных правил закодированных в корневом и внутренних узлах дерева, которые ведут поиск:

1 шаг) Поиск по длине сообщения лога(кол-во токенов сообщения)

2 шаг) Поиск по предшествующим токенам. Выбирает узел по совпавшим первым токенам лога. Причем число внутренних узлов это depth - 2, а число првоеряемых токенов тоже depth - 2; сообщение с рассматриваемым токеном состоящим из цифр переходит в специальный узел *, туда же переходят все логи в ситуации когда maxChild узла уже достигнут а сам лог не был определен ни в какого потомка

3 шаг) Поиск по схожести токенов. Когда лог попал в лист содержащий несколько групп логирований для выбора наиболее подходящей рассчитывается

$$\text{simSeq} = \frac{\sum_{i=1}^n \text{equ}(\text{seq}_1(i), \text{seq}_2(i))}{n}, \quad (1)$$

где seq_1 и seq_2 представляют соответственно сообщение лога (log message) и событие фиксированной группы логов (log event); $\text{seq}(i)$ — i -й токен последовательности; n — длина последовательностей (длина сообщения лога); функция equ определяется следующим образом:

$$\text{equ}(t_1, t_2) = \begin{cases} 1, & \text{если } t_1 = t_2, \\ 0, & \text{иначе.} \end{cases}$$

Далее, если для наибольшего simSeq среди всех групп выполняется $\text{simSeq} \geq \text{st}$, где st - порог схожести, то значит группа подобрана и ее токены будут обновлены с учетом нового лога, иначе алгоритм отмечает что подходящей группы не найдено и создается новая группа где событием event становится само сообщение лога III. Приведение лога к шаблону который соответствует листу в который попал лог; Иначе создается новая группа(лист) соответствующая сообщению лога

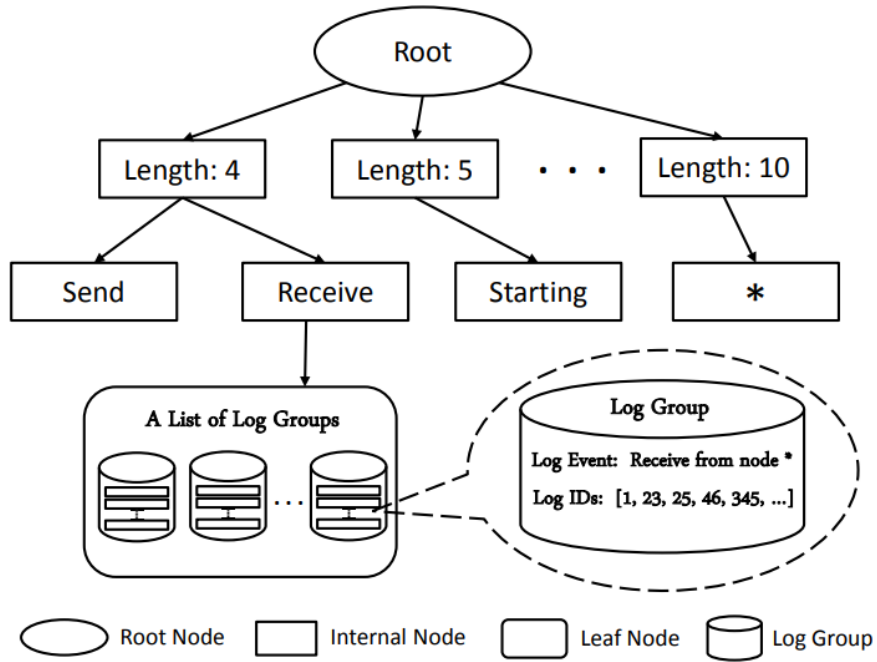


Fig. 2: Structure of Parse Tree in Drain ($\text{depth} = 3$)

Далее идет сравнение метода с другими (LKE, IPLoM, SHISO, Spell) на различных датасетах

TABLE I: Summary of Log Data Sets

System	Description	#Log Messages	Log Message Length	#Events
BGL	BlueGene/L Supercomputer	4,747,963	10~102	376
HPC	High Performance Cluster (Los Alamos)	433,490	6~104	105
HDFS	Hadoop File System	11,175,629	8~29	29
Zookeeper	Distributed System Coordinator	74,380	8~27	80
Proxifier	Proxy Client	10,108	10~27	8

$$Accuracy = \frac{2 * Precision * Recall}{Precision + Recall}, \quad (3)$$

where *Precision* and *Recall* are defined as follows:

$$Precision = \frac{TP}{TP + FP}, \quad (4)$$

$$Recall = \frac{TP}{TP + FN}, \quad (5)$$

TABLE II: Parameter Setting of Drain

	BGL	HPC	HDFS	Zookeeper	Proxifier
depth	3	4	3	3	4
st	0.3	0.4	0.5	0.3	0.3

TABLE III: Parsing Accuracy of Log Parsing Methods

	BGL	HPC	HDFS	Zookeeper	Proxifier
Offline Log Parsers					
LKE	0.67	0.17	0.57	0.78	0.85
IPLoM	0.99	0.65	0.99	0.99	0.85
Online Log Parsers					
SHISO	0.87	0.53	0.93	0.68	0.85
Spell	0.98	0.82	0.87	0.99	0.87
Drain	0.99	0.84	0.99	0.99	0.86

TABLE IV: Running Time (Sec) of Log Parsing Methods

	BGL	HPC	HDFS	Zookeeper	Proxifier
Offline Log Parsers					
LKE	N/A	N/A	N/A	N/A	8888.49
IPLoM	140.57	12.74	333.03	2.17	0.38
Online Log Parsers					
SHISO	10964.55	582.14	6649.23	87.61	8.41
Spell	447.14	47.28	676.45	5.27	0.87
Drain	115.96	8.76	325.7	1.81	0.27
Improvement	74.07%	81.47%	51.85%	65.65%	68.97%

2.4 Сравнительный анализ 18 LLM моделей: конец монополии?

ХАБР

Модель	Тип	MMLU-Pro	GPQA	HumanEval+	SWE-bench	MATH-500	AIME
DeepSeek-V3.2-Exp CN	 Open	85.0%	79.9%	—	67.8%	—	89.3%
Kimi-K2-Thinking CN	 Open	84.6%	84.5%	—	71.3%	—	94.5-100%
DeepSeek-R1 CN	 Open	84.0%	81.0%	—	49.2%	97.3%	79.8%
Mistral Large 2 FR	 Open	84.0%	—	92.0%	—	—	—
GLM-4.6 CN	 Open	~83%	81.0%	82.8%	—	—	93.9%
Qwen3-235B-A22B CN	 Open	83.0%	81.1%	—	—	—	92.3%
MiniMax-M2 CN	 Open	~82%	—	—	69.4%	—	—
Mistral Small 3 FR	 Open	81.0%	—	92.9%	—	—	—

Qwen3-30B-A3B CN	 Open	80.9%	—	—	—	—	85%
Llama 4 Maverick US	 Open	80.5%	69.8%	—	—	—	—
GPT-OSS-120B US	 Open	~80%	—	—	—	—	—
GigaChat3-10B RU	 Open	60.61%	35.02%	69.51%	—	70.0%	—
Gemma-3-12B-IT US	 Open	~55%	~35%	—	—	~70%	—
Codestral 25.01 FR	Closed	—	—	86.6%	—	—	—

Оверол рейтинг

Ранг	Модель	MMLU-Pro	GPQA	SWE-bench	Лицензия	VRAM
🏆 1	Kimi-K2-Thinking	84.6%	84.5%	71.3%	MIT	~250GB+
🥈 2	MiniMax-M2	~82%	—	69.4%	MIT	~200GB
🥉 3	GLM-4.6	—	81.0%	—	MIT	~350GB
4	Qwen3-235B-A22B	83.0%	81.1%	—	Apache 2.0	~470GB
5	DeepSeek-V3.2-Exp	85.0%	79.9%	67.8%	MIT	~700GB
6	DeepSeek-R1	84.0%	81.0%	49.2%	MIT	~700GB
7	Llama 4 Maverick	80.5%	69.8%	—	Llama	~400GB
8	GPT-OSS-120B	~80%	—	—	Apache 2.0	~80GB
9	GigaChat3-702B RU	72.76%	55.72%	—	MIT	~800GB+

ИТОГ
Выбираем **Kafka**.

3 АНАЛИЗ АНОМАЛИЙ

Идея: читать сообщения с Kafka и делать кластеризацию прогноза погоды для выявления аномальных новостей.

3.1 Данные

Для обучения понадобятся сообщения с прогнозами погоды (как допустимый класс) и, например, новости мира и различных происшествий в качестве аномальных сообщений.

Для **погодных данных** будем фетчить объекты из сайта open-meteo.com, они будут представлять из себя прогнозы погоды некоторых городов России на прошлые 30 и ближайшие 16 дней.

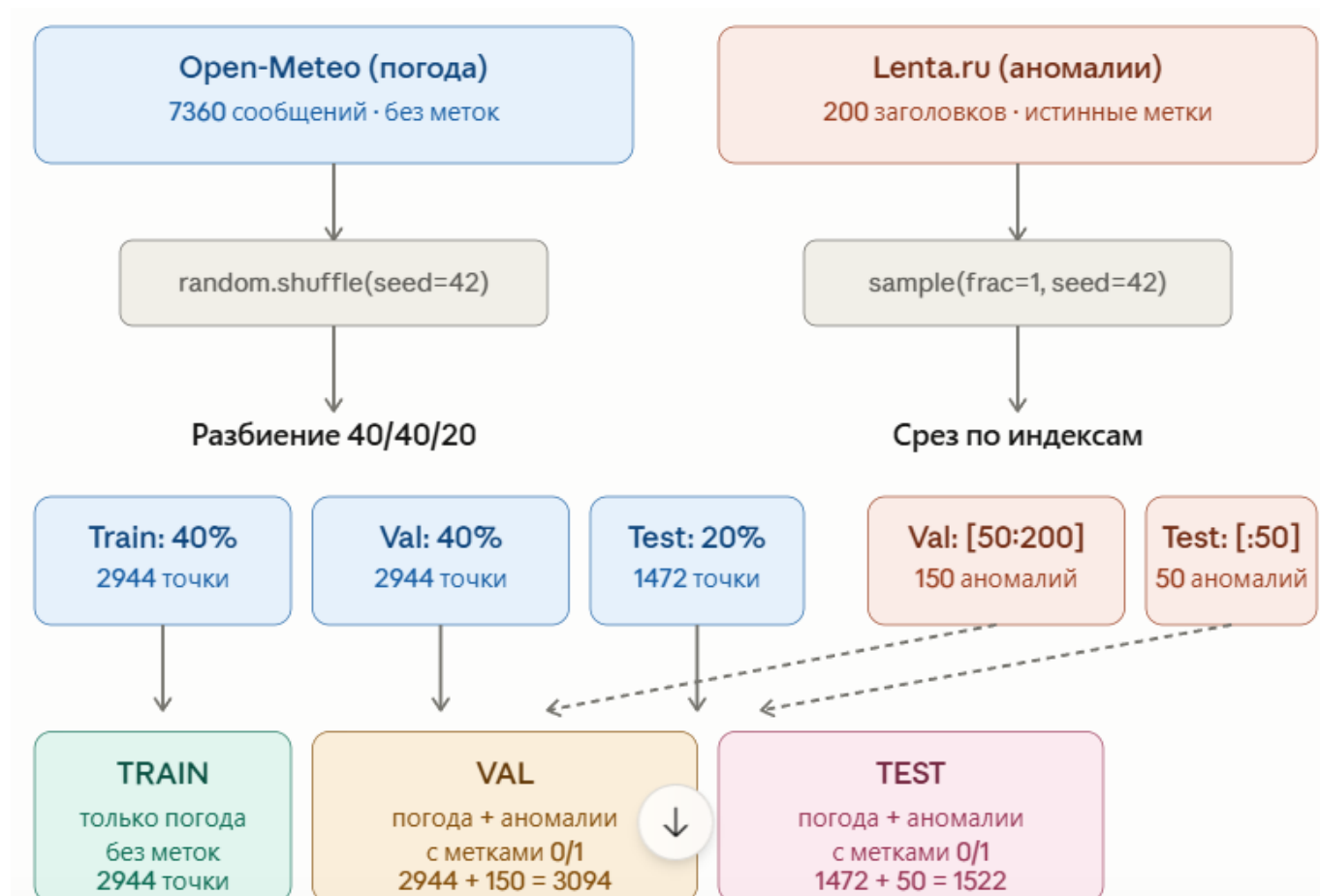
```
1 def fetch_weather_texts(cities):
2     texts = []
3     for city, (lat, lon) in cities.items():
4         url = "https://api.open-meteo.com/v1/forecast"
5         params = {
6             "latitude": lat, "longitude": lon,
7             "hourly": "temperature_2m,windspeed_10m,weathercode",
8             "forecast_days": 16,
9             "past_days": 30
10        }
11        r = requests.get(url, params=params)
12        data = r.json()
13        hourly = data["hourly"]
14
15        for i in range(0, len(hourly["time"]), 3):
16            code = hourly["weathercode"][i]
17            desc = weather_descriptions.get(code, "облачно")
18            temp = hourly["temperature_2m"][i]
19            wind = hourly["windspeed_10m"][i]
20            if temp is None or wind is None or code is None:
21                continue
22            text = f"В городе {city}: {desc}, температура {temp:.0f}°C, ветер
23                ↪ {wind:.0f} км/ч."
24            texts.append(text)
25
26    return texts
```


Для **аномалий** возьмем датасет с Kaggle.com по новостному ресурсу Lenta.ru и выберем заголовки с определенными фильтрами:

```
df = pd.read_csv("lenta-ru-news.csv")
# Аномальные – происшествия, катастрофы, политика
anomaly_df = df[df["title"].str.contains("происшествия|катастроф|чрезвычайн|война|конфликт|авария",
                                         case=False, na=False)]
```

3.2 Разбиение данных по выборкам

Разобьем данные на Тренировочную, валидационную и тестовую выборки в соотношении 40:40:20 следующим образом:



Обучение модели кластеризации будет проходить на тренировочной выборке без разметки. На валидационной выборке будут подбираться гиперпараметры модели через кроссвалидацию, поэтому валидационная состоит как из погодных сообщений (с меткой 0) так и аномальных (метка 1). Тестовая так же состоит из 2 видов сообщений и по ней будем оценивать качество модели.

3.3 Векторизация объектов

Далее представим объекты выборки в виде векторов как ансамбль эмбедингов. Для этого для каждого объекта получим 3 разных эмбедингов, сожмем каждый через **УМАР** и сконкатенируем полученное в 6 мерный вектор:

```
1 embedding_models = [model2,model3,model4]
2
3 train_embeddings_list=[]
4 val_embeddings_list = []
5 test_embeddings_list = []
6
7 for emb_model in embedding_models:
8     train_embeddings = emb_model.encode(train_weather_texts, show_progress_bar =
9         ↪ True, batch_size = 32)
10    val_embeddings = emb_model.encode(val_texts, show_progress_bar = True,
11        ↪ batch_size = 32)
12    test_embeddings = emb_model.encode(test_texts, show_progress_bar = True,
13        ↪ batch_size = 32)
14
15    train_embeddings = normalize(train_embeddings, norm='l2')
16    val_embeddings = normalize(val_embeddings, norm='l2')
17    test_embeddings = normalize(test_embeddings, norm='l2')
18
19    reducer = umap.UMAP(n_components =2,
20        n_neighbors = 15,
21        min_dist = 0.1,
22        metric = "cosine",
23        random_state=42)
24
25    train_2d_umap = reducer.fit_transform(train_embeddings)
26    val_2d_umap = reducer.transform(val_embeddings)
27    test_2d_umap = reducer.transform(test_embeddings)
28
29    train_embeddings_list.append(train_2d_umap)
30    val_embeddings_list.append(val_2d_umap)
31    test_embeddings_list.append(test_2d_umap)
32
33 TRAIN_EMBEDDINGS = np.concatenate(train_embeddings_list, axis=1)
34 VAL_EMBEDDINGS = np.concatenate(val_embeddings_list, axis=1)
35 TEST_EMBEDDINGS = np.concatenate(test_embeddings_list, axis=1)
```

В качестве эмбедингов выбраны следующие модели:

- all-MiniLM-L6-v2
- cointegrated/rubert-tiny2
- paraphrase-multilingual-mpnet-base-v2

3.4 HDBSCAN

Теперь, когда эмбединги получены, перейдем к обучению модели кластеризации.

Рассмотрим HDBSCAN: Эта модель является обобщением DBSCAN, который не зависит от формы кластеров и объявляет объекты не попавшие ни в один из кластеров аномальными.

Суть в том, что HDBSCAN запускает обучение на многих Epsilon одновременно, который зафиксирован в DBSCAN. Это позволяет отыскать наиболее устойчивые кластеры.

3.5 OPTUNA

Найдем оптимальные гиперпараметры модели HDBSCAN через фреймворк **Optuna**:

Определим пространство гиперпараметров интервалами:

```
parameters_grid = {
    "min_cluster_size": (20, 300),
    "min_samples": (1, 20),
    "cluster_selection_epsilon": (0.0, 1.0),
    "cluster_selection_persistence": (0.0, 1.0),
    "metric": ["euclidean", "manhattan"],
    "alpha": (0.5, 1.5),
    "cluster_selection_method": ["eom", "leaf"],
    "allow_single_cluster": [True, False],
}
```

Чтобы решить задачу будем подбирать гиперпараметры, на которых метрика F_1 будет максимальной.

Для этого напишем целевую функцию в которой будет реализована кроссвалидация на 3 фолдах, где каждая комбинация гиперпараметров будет обучена на двух и оценена оставшемся фолде 3 раза, где каждый фолд побывает отложенным ровно 1 раз.

```

1 def objective(trial):
2
3     parameters = {
4         "min_cluster_size": trial.suggest_int("min_cluster_size",
5         ↪ *parameters_grid["min_cluster_size"]),
6         "min_samples": trial.suggest_int("min_samples",
7         ↪ *parameters_grid["min_samples"]),
8         "cluster_selection_epsilon": trial.suggest_float("cluster_selection_epsilon",
9         ↪ *parameters_grid["cluster_selection_epsilon"]),
10        "cluster_selection_persistence": trial.suggest_float("cluster_selection_persistence",
11        ↪ *parameters_grid["cluster_selection_persistence"]),
12        "metric": trial.suggest_categorical("metric",
13        ↪ parameters_grid["metric"]),
14        "alpha": trial.suggest_float("alpha",
15        ↪ *parameters_grid["alpha"]),
16        "cluster_selection_method": trial.suggest_categorical("cluster_selection_method",
17        ↪ parameters_grid["cluster_selection_method"]),
18        "allow_single_cluster": trial.suggest_categorical("allow_single_cluster",
19        ↪ parameters_grid["allow_single_cluster"]),
20    }
21
22    fold_1 = X_train[: (len(X_train)//n_fold)]
23    fold_2 = X_train[len(X_train)//n_fold: (len(X_train) -
24    ↪ (len(X_train)//n_fold))] # здесь fold_ - тренировочные погодные данные
25    fold_3 = X_train[(len(X_train) - (len(X_train)//n_fold)):]
26
27    val_1 = X_val[: (len(X_val)//n_fold)]
28    val_2 = X_val[len(X_val)//n_fold: (len(X_val) - (len(X_val)//n_fold))] #
29    ↪ валидационные размеченные данные (погодные + аномалии)
30    val_3 = X_val[(len(X_val) - (len(X_val)//n_fold)):]
31
32    y_val_1 = y_val[: (len(y_val)//n_fold)]
33    y_val_2 = y_val[len(y_val)//n_fold: (len(y_val) - (len(y_val)//n_fold))]
34    y_val_3 = y_val[(len(y_val) - (len(y_val)//n_fold)):]
35
36    folds = [fold_1, fold_2, fold_3]
37    vals = [val_1, val_2, val_3]
38    y_vals = [y_val_1, y_val_2, y_val_3]
39
40    f1_scores = []
41
42    fold_num=0
43    for fold in folds:
44
45        train_folds = [folds[i] for i in range(len(folds)) if i != fold_num]

```

```

38     x_train = np.concatenate(train_folds, axis=0)
39
40     x_val = np.concatenate([fold, vals[fold_num]], axis=0)
41     y_vali = np.concatenate([np.zeros(len(fold),
42                               ↳ dtype=int), y_vals[fold_num]])
43
44     try:
45         clusterer = hdbscan.HDBSCAN(**parameters, prediction_data =
46                                     ↳ True).fit(x_train)
47         _, strengths = hdbscan.approximate_predict(clusterer, x_val)
48     except (IndexError, ValueError, RuntimeError) as e:
49         return 0.0
50     mask_in_cluster = clusterer.labels_ != -1
51     if mask_in_cluster.sum() < 2:
52         return 0.0
53
54     best_f1 = 0.0
55     for threshold in np.arange(0.05, 0.5, 0.05):
56         predictions = (strengths < threshold).astype(int)
57         f1 = f1_score(y_vali, predictions, zero_division=0)
58         best_f1 = max(best_f1, f1)
59     f1_scores.append(best_f1)
60
61     intermediate = np.mean(f1_scores)
62     trial.report(intermediate, fold_num) #Pruning
63     if trial.should_prune():
64         raise optuna.TrialPruned()
65
66     fold_num+=1
67
68     return np.mean(f1_scores)

```

Проведем оптимизацию на 5000 испытаний со следующими алгоритмами Сэмплера и Прунера:

TPESampler разделяет комбинации гиперпараметров на удачные и неудачные значения фиксированного гиперпараметра и присваивает им соответствующие плотности $l(\lambda)$ и $g(\lambda)$. Каждое следующее значение подбирается как

$$\lambda^* = \arg \max_{\lambda} \frac{l(\lambda)}{g(\lambda)}.$$

HyperBandPruner - отбрасывает бесперспективные комбинации экономя ресурсы.

Таблица 1: Подобранные гиперпараметры HDBSCAN

Параметр	Значение	Описание
min_cluster_size	54	Минимальный размер кластера
min_samples	3	Минимум соседей для core-точки
cluster_selection_epsilon	0,1249	Порог объединения кластеров
cluster_selection_persistence	0,0300	Порог устойчивости кластера
metric	manhattan	Метрика расстояния
alpha	0,8179	Параметр сглаживания
cluster_selection_method	eom	Метод выбора кластеров
allow_single_cluster	True	Разрешить единственный кластер

Теперь подберем порог силы на Валидационной выборке который даст на ней лучший F_1 :

```

1 _, val_strengths = hdbscan.approximate_predict(clusterer, VAL_EMBEDDINGS)
2 val_labels_arr = np.array(val_labels)
3
4 print(f"{'Попор':>8} {'TP':>4} {'FP':>4} {'FN':>4} "
5       f"{'Recall':>8} {'Precision':>10} {'F1':>6}")
6
7 best_threshold = 0.0
8 best_f1_val = 0.0
9
10 for threshold in np.arange(0.005, 0.5, 0.005):
11     pred = (val_strengths < threshold).astype(int)
12     tp_ = ((pred == 1) & (val_labels_arr == 1)).sum()
13     fp_ = ((pred == 1) & (val_labels_arr == 0)).sum()
14     fn_ = ((pred == 0) & (val_labels_arr == 1)).sum()
15
16     recall = tp_ / len(anomaly_val_texts)
17     precision = tp_ / (tp_ + fp_) if (tp_ + fp_) > 0 else 0
18     f1 = (2 * precision * recall / (precision + recall)
19          if (precision + recall) > 0 else 0)
20
21     if f1 > best_f1_val:
22         best_f1_val = f1
23         best_threshold = threshold
24
25     print(f"{threshold:>8.3f} {tp_>4} {fp_>4} {fn_>4} "
26           f"{'recall':>8.2%} {'precision':>10.2%} {'f1':>6.5f}")
27
28 print(f"\nЛучший порог по val: {best_threshold:.3f}, "
29       f"{'F1_val':>6.5f}")

```

Лучший порог по val: 0.085 давший $F_1 = 0.5176$

```

1 predicted_cluster_labels, test_strengths =
  ↳ hdbscan.approximate_predict(clusterer, TEST_EMBEDDINGS)
2 predictions = (test_strengths < best_threshold).astype(int)

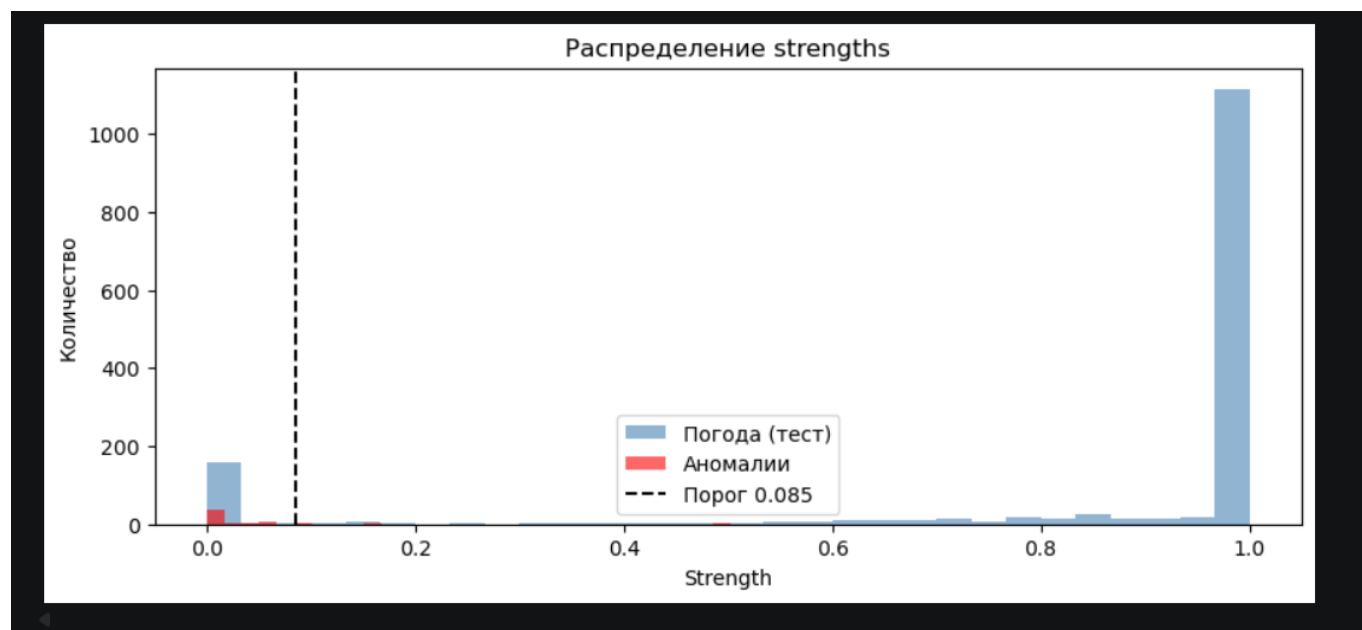
```

Оценим качество на Тестовой выборке:

Таблица 2: Результаты выявления аномалий на тестовой выборке

Метрика	Значение
True Positive (TP)	47 / 50
False Positive (FP)	163
False Negative (FN)	3
True Negative (TN)	1309
Recall	94,00 %
Precision	22,38 %
F1-score	0,3615

Ниже распределение strengths предсказаний модели на тестовой выборке



3.6 Оптимизация по $F_{1/2}$

Прделаем всё то же самое, но оптимизировав $F_{1/2}$, чтобы попытаться сместить перекоc с **recall** в сторону **precision**.

Таблица 3: Результаты выявления аномалий на тестовой выборке при оптимизации $F_{1/2}$

Метрика	Значение
True Positive (TP)	49 / 50
False Positive (FP)	367
False Negative (FN)	1
True Negative (TN)	1105
Recall	98,00 %
Precision	11,78 %
$F_{1/2}$ -score	0,1429

Такой подход ожидаемого результата не дал.

3.7 ОПТИМИЗАЦИЯ UMAP

Попробуем добавить в objective функцию оптимизацию гиперпараметров UMAP:

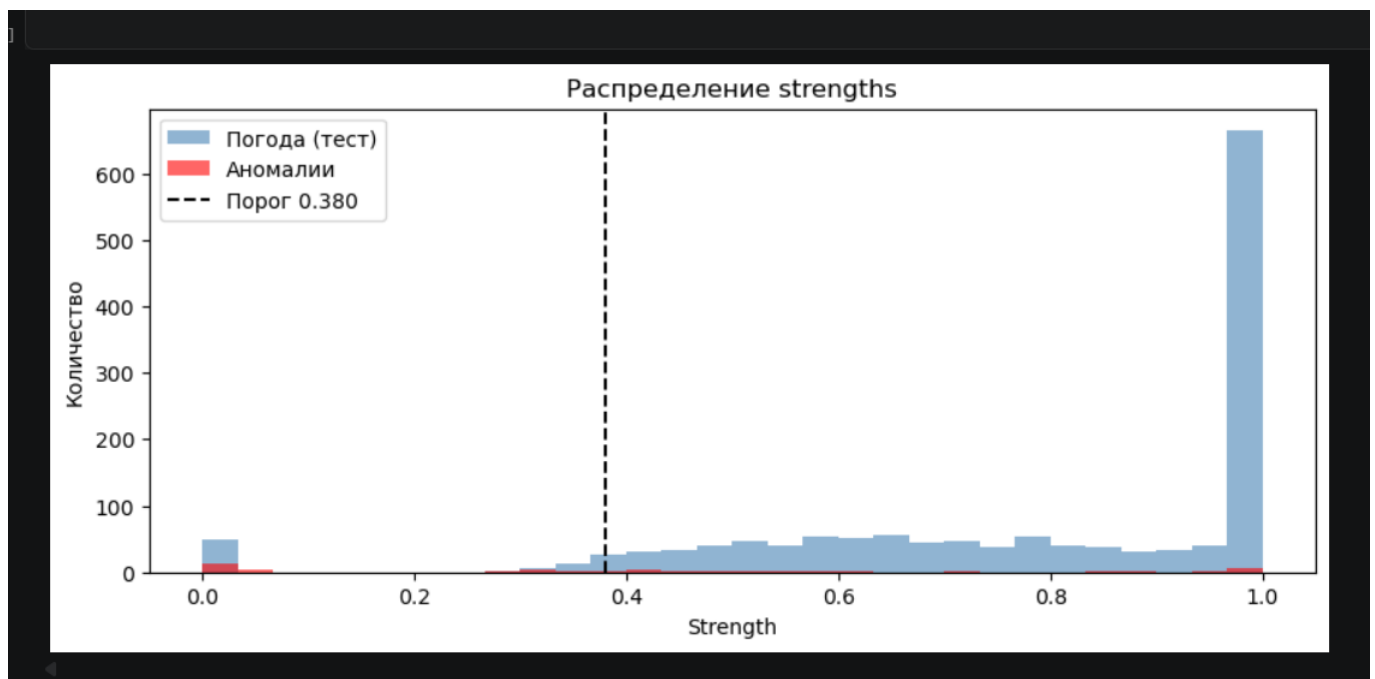
```
1  umap_grid = {
2      "n_components":      (2,6),
3      "n_neighbors":      (5,30),
4      "min_dist":          (0.0, 0.2)
5  }
6  def objective(trial):
7      umap_parameters = {"n_components": trial.suggest_int("n_components",
8          ↪ *parameters_grid["n_components"]),
9                          "n_neighbors": trial.suggest_int("n_neighbors",
10         ↪ *parameters_grid["n_neighbors"]),
11                          "min_dist": trial.suggest_float("min_dist",
12         ↪ *parameters_grid["min_dist"])}
13
14      train_reduced_list = []
15      val_reduced_list = []
16
17      for emb_train, emb_val in zip(raw_train, raw_val):
18          reducer = umap.UMAP(
19              **umap_parameters,
20              metric="cosine",
21              random_state=42
22          )
23          train_reduced_list.append(reducer.fit_transform(emb_train))
24          val_reduced_list.append(reducer.transform(emb_val))
25
26      X_train = np.concatenate(train_reduced_list, axis=1) # (N, 3*n_components)
27      X_val = np.concatenate(val_reduced_list, axis=1)
```


Таблица 4: Лучшие гиперпараметры HDBSCAN и UMAP

Параметр	Значение
<i>HDBSCAN</i>	
min_cluster_size	36
min_samples	3
cluster_selection_epsilon	0,6216
cluster_selection_persistence	0,1601
metric	manhattan
alpha	0,7017
cluster_selection_method	leaf
allow_single_cluster	True
<i>UMAP</i>	
n_components	15
n_neighbors	30
min_dist	0,0250

Лучший порог по val: 0.380, F_1 : 0.4045

Распределение strengths предсказаний HDBSCAN на тестовой выборке:



4 Доверительное оценивание F_1

В бутстрап-цикле будем случайно брать элементы тестовой выборки с возвращением для замера метрик. Затем найдём интервал, который с 95 % уверенностью покрывает истинный F_1 модели.

Таблица 6: Бутстрап-оценки метрик качества с 95 % доверительными интервалами

Метрика	Оценка	95 % ДИ
F_1	0,348	[0,273; 0,419]
Recall	0,941	[0,867; 1,000]
Precision	0,214	[0,161; 0,268]

Можно увидеть, что модель надёжно ловит аномалии, хотя так же последовательно даёт много ложных тревог.

Список литературы

- [1] Селезнёв А. И., Селезнёв И. Л. Брокеры сообщений в системах обработки данных // Белорусский государственный университет информатики и радиоэлектроники. — Минск, — С. 2.
- [2] Мусаева А. А., Григорьев Д. А. Обзор современных технологий извлечения знаний из текстовых сообщений. — С. 6.
- [3] Drain: An Online Log Parsing Approach with Fixed Depth Tree. — С. 8.
- [4] Сравнительный анализ 18 LLM моделей: конец монополии? // Хабр.